



Audyogik Shikshan Mandals Nextgen Technical Campus

Survey No. 401/1, Adjacent to Parshwa Prandyalay Jain Temple Talegaon Dabhade, Maval, Pune

www.asmnnext.edu.in / Email : nextg@asmedu.org

Affiliated to SPPU and Approved by AICTE

F.Y. M.C.A. (Management) STQ-553 MJ : SOFTWARE TESTING AND QUALITY ASSURANCE (2024 Pattern) (Semester - II)

Question Paper Solution April / May 2025

Time : 2½ Hours]

[Max. Marks : 50

Instructions to the candidates:

- 1) All questions are compulsory.
- 2) All questions carry equal marks.
- 3) Draw diagram whenever required

Question 1 (5 Marks Each) (Chapter 1)

A. Describe software quality assurance and outline its key activities.

Answer

Definition:

SQA is a systematic process of evaluating the quality of software during its development lifecycle. It ensures that the software meets the specified requirements and standards, reducing defects and improving the overall reliability of the product.

Objective:

- Make sure the work is done as per the rules and guidelines.
- Find and fix issues as soon as possible during development.
- Ensure the software works well, lasts long, and performs reliably.
- Deliver a product that meets customer expectations and earns their confidence.

SQA Activities

- **SQA Management Plan:** Create a plan to ensure quality throughout the project. Decide the best software engineering activities for the project and check if the SQA team has the required skills.
- **Set Checkpoints :** Define specific points in the project to evaluate progress. Use collected data to measure the project's performance at each checkpoint.

- **Measure Change Impact:** When fixing an error, it might cause new errors. Keep track of how changes affect the project and test the fixes to ensure compatibility with the whole system.
- **Use Multiple Testing Strategies:** Don't rely on just one testing method. Use different testing approaches to find as many issues as possible.
- **Manage Good Relations:** Build good relationships with other teams, like developers. Poor relationships can negatively affect the project. Avoid conflicts and focus on collaboration.
- **Maintain Records and Reports:** Document everything, including test cases, defects, changes, and test cycles. Share these records with stakeholders for better understanding and future reference.
- **Review Software Engineering Activities:** Check and document the processes used in the project. Make sure the software is being developed correctly and meets quality standards.
- **Formalize Deviation Handling:** Record and track anything that doesn't go as planned. Follow clear steps to address these issues in an organized way.

B. Explain the metrics used to assess and measure reliability.

Answer

Software Reliability Metrics are quantitative measures used to evaluate how reliably software performs under specified conditions for a certain period of time. These metrics help assess software quality, predict failures, and improve system performance.

Common Metrics Used to Assess and Measure Reliability

1. Mean Time To Failure (MTTF)

Measures the average operating time between software failures.

Formula

$$MTTF = \frac{\text{Total Operating Time}}{\text{Number of Failures}}$$

Example

If software runs for 1000 hours and fails 5 times:

$$MTTF = \frac{1000}{5} = 200 \text{ hours}$$

2. Mean Time To Repair (MTTR)

Measures the average time required to fix a failure and restore the software.

$$MTTR = \frac{\text{Total Repair Time}}{\text{Number of Repairs}}$$

Example

A system takes **8 hours** to repair **4 failures**.

$$MTTR = \frac{8}{4} = 2 \text{ hours}$$

Hence, average repair time is 2 hours.

Importance

- Lower MTTR means quicker recovery.
- Reflects maintainability of the system.

3. Mean Time Between Failures (MTBF)

Measures the average time between one failure and the next.

Formula

$$MTBF = MTTF + MTTR$$

Example

A system works for **200 hours** and fails **2 times**. Repair time is **4 hours**.

$$MTTF = \frac{200}{2} = 100 \text{ hours}$$

$$MTTR = \frac{4}{2} = 2 \text{ hours}$$

$$MTBF = 100 + 2 = 102 \text{ hours}$$

MTBF=100+2=102 hours

Hence, the average time between failures is 102 hours.

Importance

- Widely used reliability indicator.
- Higher MTBF means more dependable software.

4. Rate of Occurrence of Failures (ROCOF)

ROCOF measures the frequency of software failures over a specified period of operating time. It shows how often the system fails during use.

Formula

$$ROCOF = \frac{\text{Number of Failures}}{\text{Total Operating Time}}$$

Example : If an application crashes 5 times in 10 hours of operation:

$$ROCOF = \frac{5}{10} = 0.5$$

So, the ROCOF is **0.5 failures per hour**.

This means the application fails once every 2 hours. Lower ROCOF indicates a more reliable system.

5. Probability of Failure on Demand (POFOD)

POFOD measures the probability that a system will fail when a specific service or request is made. It shows how likely the system is to fail on demand.

Formula

$$POFOD = \frac{\text{Number of Failed Requests}}{\text{Total Number of Requests}}$$

Example: If a website handles 1000 requests and fails for 2 requests:

$$POFOD = \frac{2}{1000} = 0.002$$

So, the POFOD is **0.002 or 0.2%**.

This means the system fails for 2 requests out of every 1000 requests. Lower POFOD indicates a more reliable system.

6. Availability: Availability measures how much time the system is operational and accessible when required.

Formula

$$Availability = \frac{MTBF}{MTBF + MTTR}$$

Where:

- MTBF (Mean Time Between Failures) = Average working time before failure occurs.
- MTTR (Mean Time To Repair) = Average time required to repair the system after failure.

This formula shows the proportion of time the system remains operational.

Example: If MTBF = 90 hours and MTTR = 10 hours,

$$Availability = \frac{90}{90 + 10} = 0.9$$

So, Availability = 90%.

C. Explain the term quality in the context of software engineering and describe product revision factors.

Answer

In Software Engineering, **quality** means how well a software product works according to user needs and requirements. Good quality software works correctly, is reliable, easy to use, and has fewer errors.

Example: A mobile banking app that runs smoothly, gives correct balance information, and does not crash is considered a **quality software product**.

In simple words: “Quality means the software does its job properly and satisfies the user.”

Product Revision

Product Revision means improving or updating the software after it has been released. It helps the software work better, fix problems, and meet new user requirements.

1. Maintainability : Maintainability means the software should be easy to repair, update, or modify whenever errors are found or changes are required. Good maintainability saves time and cost during maintenance.

Example: If there is a mistake in attendance software, developers can quickly fix the bug without affecting the whole system.

2. Flexibility: Flexibility means the software can easily adjust to new requirements or different environments without making major changes. Flexible software can grow according to user needs.

Example: An existing Learning Management System (LMS) is updated by adding an online assignment submission feature.

3. Testability: Testability means the software should be easy to test so that defects can be identified and corrected properly. It helps ensure that all functions work correctly after updates.

Example: The login module and payment module are tested separately to check whether each feature is working properly.

D. Describe the significance of ISO 9000 standards in SQA.

Answer

The International Organization for Standardization 9000 standards are a set of international quality management standards used to ensure that organizations maintain high-quality products and services. In Software Quality Assurance (SQA), ISO 9000 helps software companies improve development processes, maintain consistency, and satisfy customer requirements.

Main Features of ISO 9000 in SQA

- 1. Quality Management System (QMS):** ISO 9000 provides guidelines to create a proper quality management system for software development.
- 2. Customer Focus:** The standards ensure that customer requirements are understood and fulfilled properly.

3. **Process-Oriented Approach:** It focuses on improving software development processes rather than only checking the final product.
4. **Documentation and Standards:** Proper documentation of coding, testing, maintenance, and project management is maintained.
5. **Continuous Improvement:** Organizations regularly improve their software processes and quality practices.
6. **Error Prevention:** ISO 9000 emphasizes preventing defects before they occur instead of correcting them later.

Important ISO 9000 Standards

- **ISO 9001:** This is the most widely used standard for quality management systems. It defines requirements for maintaining software quality and customer satisfaction.
- **ISO 9000:** It explains the basic concepts, principles, and vocabulary related to quality management systems.
- **ISO 9004:** It provides guidelines for improving organizational performance and achieving long-term success.

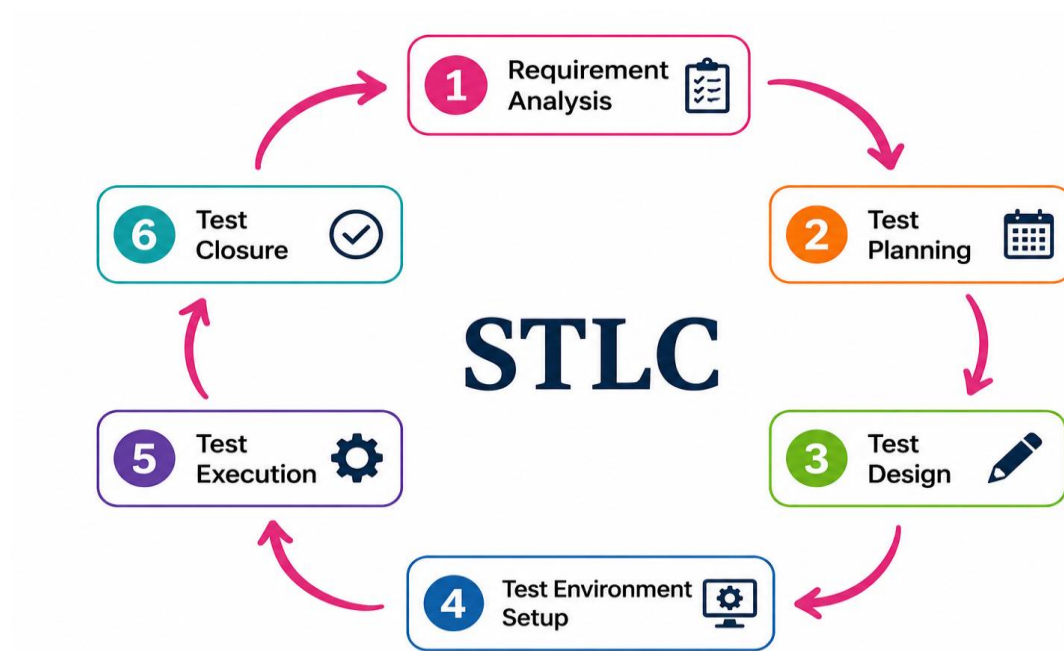
Benefits of ISO 9000 in SQA

- Improves software quality
- Increases customer satisfaction
- Reduces software defects
- Enhances process efficiency
- Improves documentation and management
- Builds customer trust and company reputation

Question 2 (5 Marks Each) (Chapter 2)

A. Outline the steps in software testing life cycle, highlighting the sequence of testing phases.

Answer



1. Requirement Analysis :

Requirement Analysis is the first step of the Software Testing Life Cycle (STLC). In this phase the quality assurance team understands the requirements like what is to be tested. If anything is missing or not understandable then the quality assurance team meets with the stakeholders to better understand the detailed knowledge of requirements.

Example: A company is developing an **e-commerce website**. The testing team analyzes requirements such as user login, product search, and payment gateway to identify testable scenarios.

2. Test Planning:

The test strategy is prepared, and the scope of testing is defined. The required tools, resources, and timelines are decided. This phase gets started once the requirement-gathering phase is completed.

Example: The test manager creates a plan that includes the number of testers, test environment setup, manual or automation testing, and estimated testing time.

3. Test Case Development (Test Design):

The test case development phase gets started once the test planning phase is completed. In this

phase the testing team notes down the detailed test cases. The testing team also prepares the required test data for the testing. When the test cases are prepared then they are reviewed by the quality assurance team.

4. Test Environment Setup :

Setup of the test environment is an independent activity and can be started along with Test Case Development.

Test Environment Setup is the process of preparing the hardware, software, network, and test data required for executing test cases. This phase includes configuring servers, databases, test tools, and necessary system dependencies

Proper environment setup helps in detecting and resolving issues early, ensuring smooth test execution.

In this process, the testing team is not involved. either the developer or the customer creates the testing environment.

5. Test Execution:

After the test case development and test environment setup, the test execution phase gets started. In this phase the testing team starts executing test cases based on prepared test cases in the earlier step. The test cases are executed, and any defects found are reported to the development team.

6. Test Closure

Once testing is completed, a final report is prepared, summarizing the testing process, defects found, and lessons learned.

Example: The test team creates a **Test Summary Report** showing:

- Total test cases executed
- Number of defects found and fixed
- Overall software quality assessment

After final approval, the e-commerce website is ready for release.

B. Differentiate between fault, defect and failure and describe the common causes of software failure

Answer

Basis	Fault	Defect	Failure
Meaning	A fault is a mistake in the program code made by the developer.	A defect is an issue found in the software during testing due to a fault.	A failure occurs when the software does not work as expected during execution.
Occurs In	Occurs during coding or development.	Occurs during testing of the software.	Occurs when the software is used by the user.

Identified By	Usually identified by developers.	Usually identified by testers or QA teams.	Usually identified by end users while running the application.
Example	Wrong formula written for calculating marks.	Attendance is calculated incorrectly in the application.	The application crashes when a student logs in.

Common Causes of Software Failure

- **Poor Requirement Understanding:** If developers do not clearly understand customer requirements, the software may not meet user needs.
Example: An online exam system is developed without a timer feature required by users.
- **Coding Errors:** Mistakes in programming logic or syntax can cause software defects and failures.
Example: Incorrect coding causes wrong calculation of student grades.
- **Lack of Proper Testing:** If software is not tested properly, many bugs remain undetected before release.
Example: Payment module fails because it was not fully tested.
- **Communication Gap:** Poor communication between developers, testers, and clients can create misunderstandings.
Example: The client requests a mobile-friendly design, but developers create only a desktop version.
- **Changing Requirements:** Frequent changes in requirements during development may create confusion and errors.
Example: New features are added at the last moment, causing system instability.
- **Hardware or Environment Issues:** Software may fail because of incompatible hardware, network problems, or system configuration issues.
Example: Software works on Windows but crashes on Linux systems.

C. Illustrate the concept of performance testing and detail any three specific performance test categories

Answer

Performance Testing

Performance Testing is a type of software testing used to evaluate the speed, responsiveness, stability, and efficiency of a software application under different workloads. It helps determine how well the system performs when many users access it simultaneously. The main purpose of performance testing is to identify performance bottlenecks and ensure the software provides a smooth user experience.

For example, an online shopping website should load quickly and work properly even during heavy traffic on festival sales.

Types of Performance Testing

1. Load Testing

Load testing checks how the software works under normal or expected number of users. It helps to know whether the system works smoothly without slowing down.

Example: Checking a banking application when 5,000 users log in at the same time.

2. Stress Testing

Stress testing checks how the software behaves under very heavy load or extreme conditions. It helps find the limit of the system and whether it crashes or recovers properly.

Example: Testing an online shopping website during a big sale when millions of users visit together.

3. Spike Testing

Spike testing checks how the software handles sudden increases or decreases in users. It ensures the system can manage unexpected traffic changes.

Example: A ticket booking website suddenly gets thousands of users when booking starts.

D. Discuss the Integration test approaches.

Answer

Integration Testing Approaches

Integration Testing means checking whether different modules of software work properly together. It helps find communication problems between modules. This testing is done after unit testing. It ensures that combined modules give correct output.

1. Big Bang Approach

In this approach, all modules are connected together and tested at one time. It is easy for small software projects but difficult for large systems. Finding errors becomes hard because all modules are tested together. This approach needs all modules to be completed before testing starts.

Example: All modules of a college management system are tested together after development.

2. Top-Down Approach

In this approach, testing starts from the top modules and then moves to lower modules. Missing lower modules are replaced by stub programs. It helps check the main functions first. Errors in major control functions can be found early.

Example: Testing the main menu first and later testing submenu options.

3. Bottom-Up Approach

In this approach, testing starts from lower modules and moves upward. Driver programs are used for testing higher modules. It helps test basic functions first. This approach is useful when lower modules are very important.

Example: Testing the database module before testing the user interface.

4. Sandwich (Hybrid) Approach

This approach combines both Top-Down and Bottom-Up methods. Testing is done from both top and bottom sides together. It saves time and improves testing speed. It is useful for large software systems.

Example: Testing login page and database modules at the same time.

Question 3 (5 Marks Each) (Chapter 3)

A. In an examination, a candidate must score at least 24 marks to pass, with the maximum possible score being 40 marks. Using the Equivalence Partitioning technique:

- i) Identify the equivalence partitions for the marks field.
- ii) Determine the minimum number of test cases required to effectively test the field.
- iii) Specify the test cases, including representative values and their expected outcomes

Answer

Given:

Passing Marks =24

Maximum Marks =40

i) Identify the Equivalence Partitions

We divide input marks into valid and invalid groups

Valid Partitions

- 24 to 40 —> Pass

Invalid Partitions

- Marks < 0 = Invalid Marks
- 0 to 23 = Fail
- Marks > 40 = Invalid Marks

ii) Minimum Number of Test Cases Required

Minimum **4 test cases** are required to cover all equivalence partitions.

iii) Test Cases with Representative Values and Expected Outcomes

Test Case	Representative Value	Expected Outcome
TC1	-5	Invalid Marks
TC2	20	Fail

TC3	30	Pass
TC4	45	Invalid Marks

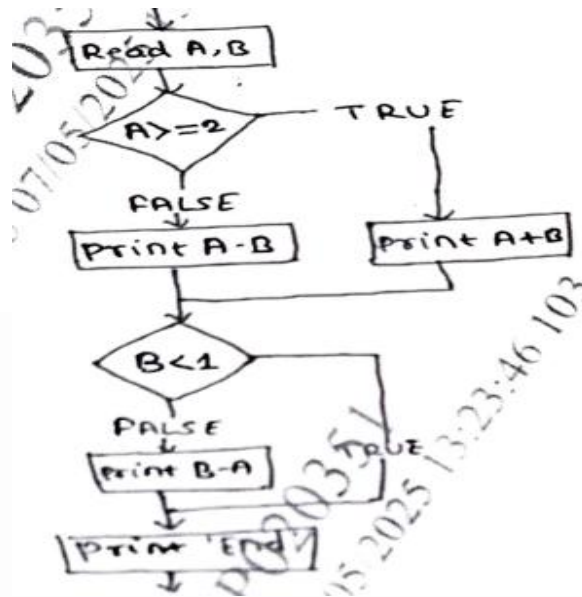
B. Differentiate between static and dynamic testing.

Answer

Aspect	Static Testing	Dynamic Testing
Meaning	Static Testing is a type of testing where the code is checked without executing the program.	Dynamic Testing is a type of testing where the program is executed to find errors.
Execution	In Static Testing, the program is not run.	In Dynamic Testing, the program is run.
Focus	Static Testing focuses on finding errors in code, design, and documents.	Dynamic Testing focuses on checking the behavior and output of the software.
When Performed	Static Testing is done in early stages of development.	Dynamic Testing is done after code is developed.
Techniques	Static Testing includes reviews, walkthroughs, and inspections.	Dynamic Testing includes unit testing, integration testing, system testing.
Errors Found	It finds syntax errors, design issues, and documentation errors.	It finds runtime errors, logical errors, and performance issues.
Who Performs	Static Testing is performed by developers and reviewers.	Dynamic Testing is performed by testers and developers.
Example	Example: Reviewing code or documents without running the program.	Example: Running a program and checking if output is correct.

C. Given the following flow chart diagram:

What is the minimum number of test cases required for 100% Statement Coverage 100% Decision Coverage Justify your answer.



Answer

1. Statement Coverage (100%)

Statement Coverage means every statement in the flowchart should execute at least once.

The flowchart has:

- Print A+B
- Print A-B
- Print B-A
- Print "End"

To execute all statements, we need:

- One test case where $A \geq 2 \rightarrow$ executes Print A+B
- One test case where $A < 2$ and $B < 1 \rightarrow$ executes Print A-B and Print B-A

Minimum Test Cases for 100% Statement Coverage = 2

Test Cases

Test Case	A	B	Output
TC1	3	2	Print A+B
TC2	1	0	Print A-B, Print B-A, End

2. Decision Coverage (100%)

Decision Coverage means every decision should have both TRUE and FALSE outcomes tested.

Decisions in flowchart:

1. **A >= 2**
 - TRUE
 - FALSE
2. **B < 1**
 - TRUE
 - FALSE

To cover all TRUE and FALSE conditions, we need 3 test cases.

Minimum Test Cases for 100% Decision Coverage = 3

Example Test Cases

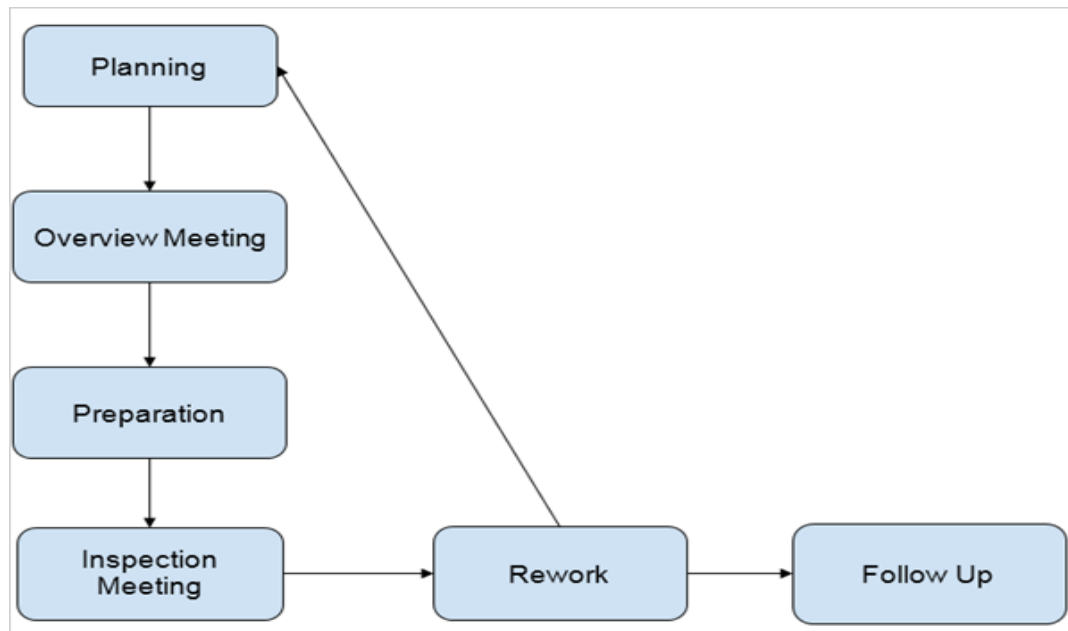
Test Case	A	B	Decision Covered
TC1	3	2	A >= 2 → TRUE
TC2	1	0	A >= 2 → FALSE, B < 1 → TRUE
TC3	1	2	A >= 2 → FALSE, B < 1 → FALSE

- Minimum test cases for 100% Statement Coverage = 2
- Minimum test cases for 100% Decision Coverage =

D. Discuss inspection process detail

Answer

Inspection is a formal process used to carefully check software documents or code to find mistakes. It's like a detailed group review where a team looks for errors in things like design plans, requirements, or code — **without running the program**.



1. Planning:

- The moderator organizes the inspection by defining the scope, goals, and schedule.
- Documents are shared with participants, and checklists may be provided to guide the review.

2. Overview: The author gives a brief explanation of the work being reviewed, helping participants understand the context.

3. Preparation:

- Each participant independently examines the document or code.
- They use checklists to identify defects like logic errors, security flaws, or non-compliance with standards.

4. Inspection Meeting:

- The team discusses the defects they found.
- The scribe records all identified issues but doesn't suggest solutions — the focus is purely

on finding problems.

5. Rework: The author fixes the defects based on the feedback provided.

6. Follow-up: The moderator verifies whether the corrections were made and decides if a re-inspection is needed.

Who's involved / Roles in the Inspection Process:

- **Author:** The person who wrote the code or document.
- **Moderator:** Keeps the review on track and organized.
- **Reviewer/Inspector :** Team members who look for mistakes.
- **Scribe/ Recorder :** Writes down the issues found during the meeting.



Question 4 (10 Marks Each) (Chapter 4)

A. Cloud kitchen aggregator app “Smoke & Spice” lists and promotes cloud kitchens, making them accessible to customers for online food ordering. It enables cloud kitchens to manage their menus, orders, and promotions while integrating with third-party delivery partners and payment gateways. The app challenges are handling high-order volumes, real-time order updates, seamless third-party integrations, and secure transactions.

Write a test plan for the following sections of the IEEE 829 test plan template for the above application:

- i) Scope of testing
- ii) Objectives
- iii) Risks
- iv) Strategy
- v) Approach

Answer (Update your answer as per marks this include overall you can reduce point as per convenience)

i) Scope of Testing (What we will test and what we will NOT test)

What we WILL test:

- **Customer mobile app** – searching for cloud kitchens, viewing food menus, adding items to cart, applying promo codes, placing orders, tracking order status in real time.
- **Kitchen dashboard (cloud kitchen panel)** – updating food menu (add/edit/delete items), marking orders as "preparing", "ready for pickup", "dispatched", "delivered", creating discount promotions, viewing order history.
- **Order management system** – handling multiple orders at the same time, cancelling orders before preparation, auto-assigning orders to delivery partners.
- **Payment gateway integration** – testing UPI, credit/debit cards, net banking, wallets; testing success, failure, partial payment, refund scenarios.

- **Third-party delivery partner integration** – sending order details to delivery apps (like Zomato, Swiggy, Dunzo), receiving live tracking updates, handling delivery partner assignment and rejection.
- **Real-time updates** – order status changes should reflect instantly on customer app (e.g., "Order confirmed" → "Preparing" → "Out for delivery").
- **Performance under high load** – app should work smoothly when 1000+ customers order during lunch/dinner time.
- **Security testing** – customer login, payment details, order data should be encrypted; no unauthorized access.

What we will NOT test:

- Physical taste or quality of food (this is the kitchen's responsibility, not app testing).
- Delivery partner's internal systems (like their driver app's GPS bugs).
- Kitchen hardware (ovens, printers, etc.).
- Manual phone orders (only app-based orders are in scope).

ii) Objectives (What we want to achieve by testing)

1. **Correct order placement** – Every customer should place an order without errors. Cart total must match final payment.
2. **Real-time updates** – When the kitchen changes order status, the customer sees it within 2 seconds without refreshing.
3. **Payment security** – Payment details must be encrypted. Failed payments should show simple error messages (not technical errors).
4. **Smooth third-party integration** – Delivery partner gets correct customer details. If the delivery partner rejects, app finds another partner automatically.
5. **Handle high traffic** – App must work properly when 500+ users order at the same time (no crash or slow down).

6. **Proper error handling** – If payment fails, order is not created. If the delivery API is down, show a friendly message like "Please try again later".

iii) Risks (What can go wrong during testing or after launch)

Technical risks:

- **Third-party API downtime** – Payment gateway or delivery partner server may be unavailable during testing or peak hours, causing order failures.
- **Real-time update delay** – WebSocket or push notification service may lag, so kitchen updates order status but customer sees old status.
- **Database overload** – During high order volume, database queries may become slow, causing order duplication or missing orders.
- **Security vulnerability** – Hackers may try to steal payment data or change order amounts using API manipulation.

Project risks:

- Time shortage – Testing all integrations (multiple payment methods + multiple delivery partners) may take longer than planned.
- Test environment mismatch – Test servers may behave differently than live production servers, hiding some bugs.
- Unclear requirements – Business rules for promotions or cancellation may keep changing, leading to re-testing.

Operational risks:

- Low kitchen cooperation – Cloud kitchens may not provide test accounts or test menu data on time.
- Delivery partner not available for testing – Live testing with real delivery partners may require extra approvals.

v) Strategy (How we will test – which types of testing)

We will use different testing methods at different stages:

Testing Type	What we will do
Unit Testing	Developers test small pieces like "add to cart function" or "apply promo code function" before integration.
Integration Testing	Test connection between app → payment gateway, app → delivery partner, app → kitchen dashboard. Check if data flows correctly.
System Testing	Test complete end-to-end flow: customer searches food → adds to cart → pays → kitchen receives order → kitchen updates status → delivery partner picks → customer receives.
UI Testing	Check that buttons, screens, fonts, colors work properly on Android and iOS devices.
Real-time Testing	Manually change order status from kitchen panel and measure how fast customer app updates.
Performance/Load Testing	Use JMeter or LoadRunner to simulate 500, 1000, 2000 users ordering at the same time. Check response time and error rate.
Security Testing	Check for SQL injection, broken authentication, exposed payment APIs. Use OWASP guidelines.
Regression Testing	After fixing a bug, re-test old features to ensure nothing else broke.
User Acceptance Testing (UAT)	Let real cloud kitchen owners and real customers test the app before final launch.

Test Environment:

- Separate test server with dummy data.
- Test payment cards (Visa, Mastercard, UPI mock).
- Mock delivery partner simulator for initial testing.

Test Tools:

- Jira for bug tracking.
- Postman for API testing.

- JMeter for load testing.
- Appium for mobile automation (optional).

v) Approach (Step-by-step how we will test)

Step 1: Planning – Prepare test cases and test data (dummy kitchens, dishes, promo codes, customer accounts).

Step 2: Environment setup – Deploy app on test server, setup test payment gateway, connect to delivery partner sandbox.

Step 3: Functional testing – First test positive flows (order → pay → deliver). Then test negative flows (payment fail, delivery reject, wrong promo).

Step 4: Real-time testing – Kitchen changes order status. The customer sees an update within 2 seconds without a refresh.

Step 5: Integration testing – Test all payment methods (success/fail/refund). Test delivery API (send order → get tracking).

Step 6: Performance testing – Test with 500 users ordering together. Check response time (<3 sec) and error rate (<1%).

Step 7: Security testing – Try to change order amount or access another user's data – system must reject.

Step 8: Bug tracking – Log bugs in Jira. Developer fixes → tester retests → close bug.

Step 9: Sign-off – All critical bugs fixed, 95% tests passed, performance and security tests passed.

Step 10: Post-launch test – After live launch, retest 10 main features to confirm nothing broke.

B. Write suitable test case for the above application.

Test Case ID	Test Scenario	Precondition	Test Steps	Expected Result
TC01	User Registration	User is on registration page	Enter valid details and click register	User account should be created successfully
TC02	User Login	User account exists	Enter valid username and password	User should login successfully
TC03	Invalid Login	User account exists	Enter wrong password	Error message should display
TC04	View Menu	User logged in	Open restaurant menu	Food items should display properly
TC05	Add Item to Cart	Food menu available	Select food item and click add to cart	Item should be added to cart
TC06	Place Order	Items available in cart	Click place order button	Order should be placed successfully
TC07	Payment by UPI/Card	Order created	Enter payment details and pay	Payment should complete successfully
TC08	Invalid Payment	Order created	Enter invalid payment details	Payment failure message should display
TC09	Order Tracking	Order already placed	Open order tracking page	Live order status should display
TC10	Coupon Code Apply	Valid coupon available	Enter coupon code during checkout	Discount should apply successfully
TC11	Third-Party Delivery Integration	Delivery service connected	Assign delivery partner	Delivery details should update correctly
TC12	High Traffic Performance	Multiple users active	Many users place orders together	App should work without crashing
TC13	Logout	User logged in	Click logout button	User should logout successfully

TC14	Menu Update by Kitchen	Kitchen admin logged in	Add or update food item	Updated menu should display to users
TC15	Notification Check	Order status changed	Trigger order update	User should receive notification

Question 5 (5 Marks Each) (Chapter 5)

A . Define CAST and explain its benefits in software testing.

Answer

CAST (Computer Aided Software Testing)

CAST means using special software tools to help in software testing. These tools make testing easier, faster, and more accurate. Instead of checking everything manually, testers use CAST tools to run test cases, find errors, and prepare reports automatically.

CAST is useful in testing websites, applications, and software systems. It helps testers save time and improve software quality.

Benefits of CAST in Software Testing

- **Saves Time:** CAST tools can run many test cases automatically in a short time. This makes the testing process faster than manual testing. It helps organizations complete projects within deadlines.
- **Reduces Human Errors:** Manual testing may include mistakes due to tiredness or carelessness. CAST tools perform testing accurately and reduce human errors. This increases the reliability of test results.
- **Improves Software Quality:** The tools check more functions and identify defects early, which helps in improving the quality and reliability of software. Better quality software increases customer satisfaction.
- **Reuse of Test Cases:** Once test scripts are created, they can be used again whenever needed, especially during regression testing. This reduces repeated manual work for testers.
- **Better Test Coverage:** CAST tools can test large amounts of data and different conditions, providing wider test coverage. It ensures that most parts of the software are properly tested.
- **Generates Reports Automatically:** These tools create test reports and defect reports automatically, making it easy to track testing progress. Reports help managers understand

software quality clearly.

- **Useful for Regression Testing:** When software is updated, CAST tools quickly retest old features to ensure they still work properly. This helps maintain stability in the software system.
- **Increases Productivity:** Testers can complete more work in less time, which increases overall productivity of the testing team. It allows testers to focus on complex testing tasks.
- **Supports Continuous Testing:** CAST tools are useful in Agile and DevOps environments where software testing is done continuously during development. This helps in faster software delivery and quick feedback.

B . Describe the step Introducing a testing tool into an organization

Answer

Successfully introducing and integrating a testing tool into an organization's workflow requires proper planning, training, and continuous improvement. The following steps help in successful integration:

Key Steps Involved

1. **Identify the Need :** First, the organization identifies a problem or requirement where the tool can help, such as improving testing, communication, or project management. This step ensures that the tool is actually needed and will solve a real problem. It helps avoid unnecessary investment.
2. **Tool Selection :** Different tools are evaluated based on cost, features, compatibility, and ease of use. The best-suited tool is selected. Comparison of multiple tools helps in making the right decision. Proper selection increases efficiency and reduces future issues.
3. **Planning and Preparation :** A proper plan is created for implementation, including budget, timeline, and resources required. Roles and responsibilities are also clearly defined. This ensures smooth execution without confusion.
4. **Installation and Setup:** The tool is installed and configured according to the organization's needs. System requirements and settings are properly adjusted. This step makes the tool ready for actual use.
5. **Training Employees :** Staff members are trained so they can use the tool effectively and confidently. Training sessions, demos, or manuals may be provided. It reduces errors and

increases productivity.

6. **Pilot Testing :** The tool is tested on a small scale to check if it works properly before full implementation. Any issues or bugs are identified and fixed at this stage. It reduces risk before full deployment.
7. **Full Implementation:** After successful testing, the tool is deployed across the organization. All users start using the tool in a real work environment. This step marks complete adoption of the tool.
8. **Monitoring and Support :** Performance is monitored, and technical support is provided to resolve issues. Regular updates and maintenance are also carried out. It ensures long-term

C . Explain the purpose of postman in API Testing

Answer

Postman is a popular API testing tool used by developers and testers to test, develop, and manage APIs easily. It provides a simple graphical interface to send requests to the server and view responses without writing complex code.

API testing checks whether APIs are working correctly, securely, and efficiently. Postman helps testers perform this testing quickly and accurately.

1. Sending API Requests

Postman allows testers to send different HTTP requests such as GET, POST, PUT, and DELETE. It helps check whether the API is working correctly. Testers can test APIs without writing complex code.

2. Validating API Responses

Postman shows API responses including status codes, response time, and returned data. This helps testers verify whether the output matches the expected result. Errors can also be identified easily.

3. Automation of Testing

Postman supports automated API testing using test scripts and collections. Repeated tests can be executed quickly without manual effort. This is useful for regression testing.

4. User-Friendly Interface

Postman provides a simple and easy graphical interface for API testing. Beginners can understand and use the tool easily. It simplifies the testing process for developers and testers.

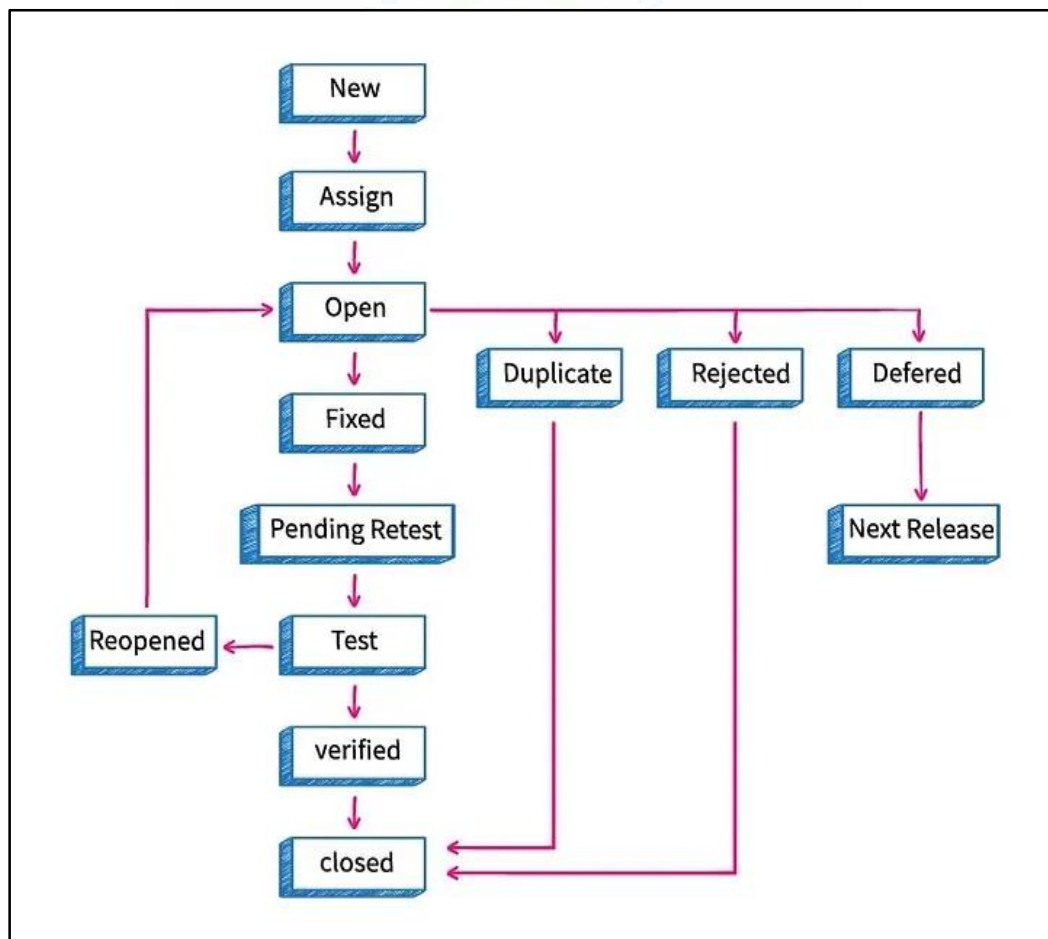
5. Collection and Environment Management

Postman allows users to save API requests in collections for future use. It also supports environment variables for testing different setups. This improves organization and efficiency in testing.

6. Collaboration and Sharing

Teams can share collections, test cases, and API documentation using Postman. This improves communication between developers and testers. It also helps maintain consistency in testing activities.

D. Describe Defect Life Cycle



1. **New** – A defect is identified and recorded for the first time. It is logged into the defect tracking system with all details. No action is taken yet at this stage.

2. **Assign** – The defect is assigned to a developer or team member. Responsibility for fixing the issue is clearly defined. The developer reviews the defect details before starting work.
3. **Open** –The developer begins working on the defect.The issue is analyzed to find the root cause.Necessary changes are planned to fix it.
4. **Fixed** –The developer corrects the defect in the code.Required changes are implemented and updated. The defect is now ready for testing.
5. **Test** –The tester checks whether the defect is fixed properly. Different test cases are executed to validate the fix. The tester ensures no new issues are introduced.
6. **Verified** – The tester confirms that the defect is resolved successfully. The fix works as expected in all scenarios.The defect is ready to be closed.
7. **Closed** – The defect is completely resolved and finalized. No further testing or changes are required. It is marked as completed in the system.
8. **Reopened** –If the defect still exists, it is opened again. The previous fix did not work correctly. It is sent back to the developer for correction.
9. **Duplicate** –The defect is already reported earlier. It matches an existing defect in the system. So, it is marked as duplicate to avoid repetition.
10. **Rejected** – The defect is not valid or cannot be reproduced. It may be due to misunderstanding or incorrect data. So, it is not considered for fixing.
11. **Deferred** –The defect is valid but not fixed immediately. It is postponed due to low priority or time constraints. It will be handled in future if needed.
12. **Next Release** – The defect is planned to be fixed in the next version.It is not critical for the current release. It will be resolved in upcoming updates.